The Journal of Systems and Software 100 (2015) 91–102

Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Profiling and classifying the behavior of malicious codes

Mamoun Alazab

Australian National University, Canberra, Australia

ARTICLE INFO

Article history:
Received 9 October 2013
Received in revised form 5 September 2014
Accepted 19 October 2014
Available online 25 October 2014

Keywords:
Cybercrime
Malware
Profiling

ABSTRACT

Malware is a major security threat confronting computer systems and networks and has increased in scale and impact from the early days of ICT. Traditional protection mechanisms are largely incapable of dealing with the diversity and volume of malware variants which is evident today. This paper examines the evolution of malware including the nature of its activity and variants, and the implication of this for computer security industry practices.

As a first step to address this challenge, I propose a framework to extract features statically and dynamically from malware that reflect the behavior of its code such as the Windows Application Programming Interface (API) calls. Similarity based mining and machine learning methods have been employed to profile and classify malware behaviors. This method is based on the sequences of API sequence calls and frequency of appearance.

Experimental analysis results using large datasets show that the proposed method is effective in identifying known malware variants, and also classifies malware with high accuracy and low false alarm rates. This encouraging result indicates that classification is a viable approach for similarity detection to help detect malware. This work advances the detection of zero-day malware and offers researchers another method for understanding impact.

1. Introduction

Cybercrime is growing and rapidly adapts to new opportunities in cyberspace. The Internet makes it easy for cyber criminals to operate virtually from abroad and remain anonymous online. It is therefore challenging to identify the source of the crime such as malware. Advanced technologies such as The Onion Router (the ability to apply multiple proxy routing that inhibits the ability to trace-back), Obfuscation (a term used to describe modification of a program to disguise its purpose, see details in Section 2), Dynamic Domain Name System (DNS), and Virtual Private Network (VPN) services are all capable of facilitating cybercrime. These methods enable cyber criminals to further their criminal operations by developing new forms and/or variants of malware to assist in the propagation of malware, which enables offenders to more effectively achieve online anonymity, to avoid network surveillance, and traffic analysis by regulators. When investigating cybercrime there are basic questions that arise. *Where* and *when* was the crime committed? *What* technologies were involved? *How* was the crime committed? *Who* committed the crime and *why* was the crime committed? Most research focuses on one aspect and neglects others and this maybe that practitioners have limited time to address problems and/or little or no experience with the wider problems caused by the Advanced Persistent Threat (APT) in cybercrime. A learning profiling approach could be helpful when time is limited and consequences are critical. Efforts by law enforcement and computer analysts working in government and non-government have helped to suppress cybercrime activities (Cowley, 2012; Anderson et al., 2013; Broadhurst et al., 2013). Most effective have been efforts that have focused on the analysis of the technical aspects of Internet crime, for example, deciphering malicious codes, identifying malware (Alazab and Venkatraman, 2013), shutting down botnets and other methods used by cybercriminals, and designing increasingly sophisticated strategies to protect computer systems. At the same time, a general lack of knowledge of the perpetrators themselves, their motivation, the development of new malware code, and their *modus operandi*, leads to many failures in the suppression of cybercrime and, therefore, cybercrime remains a fast growing global threat (Alazab et al., 2013).

The majority of malicious software is recycled and has not been written from scratch. In 2014, Symantec noted (Symantec Corporation, 2014) that the number of genuinely new malware families created has slowed as malware coders are working to perfect existing malware. Indeed, some malicious codes are being reused and modified. For example, in 2010 Symantec detected more than 286 million new malware variants (Symantec, 2011)

E-mail address: mamoun.alazab@anu.edu.au

http://dx.doi.org/10.1016/j.jss.2014.10.031
0164-1212/

including 90,000 unique variants of the Zeus toolkit and, more recently, over 20 million variants were found in 2013 alone (Dell, 2014). The similarity between many botnet variants like Hlux, Waledac, Nuwar, Kelihos and Storm suggest that these bots were developed by the same botnet coders (details in Bureau, 2011). This year the Fox-IT InTEL technical report (Fox-IT InTELL, 2014) verified that the malware family Tilon was also linked with Zeus and the SpyEye malware family. It can be argued that malware codes have sufficiently distinct features from each other that could allow for profiling and further analysis. The lack of a comprehensive analysis of malware – not only for the purpose of detection – makes it hard to identify novel forms of attack, characterize the method of attack, and predict similar attacks in the future.

Behavior reflects personality and in the online world it reflects offenders. Profiling is an investigative tool that consists of analyzing the crime scene and likely behavior of the offender (in this case malware code) and using the information from this to determine the probable identity of the cybercriminal. Profiling criminal types may be derived from inductive methods that draw on the statistically derived characteristics of offenders from resolved cases, or by applying more speculative deductive methods based on the assumption that the *modus operandi* of the crime will be repeated in the same way by the same offender – in short the crime will have a signature (Douglas et al., 1986). Profiling in criminology is not a new method and the concept has been deployed with varying success in solving crimes, especially crimes that are violent and may show signs of offender pathology. Malware profiling is relatively new as a method for crime analysis. It is difficult to build the right profile for each and every malware code simply because malware codes can be programmed by someone, adjusted by another, and used by another. The motives and the circumstances should always be considered when a profile of malware code is assembled and so a holistic approach is required. The role of an approach similar to the notion of behavior profiling could be a promising approach, although there are many problems involved in successfully distinguishing legitimate behavior from malicious ones in the cybercrime context. To use a profiling in the Windows Operating System (OS) required close understanding of the hardware and software of the computer system.

The focus of this paper is to explore the properties and characteristics of the Portable Execution (PE) files. Similarity based detection of malware has been proposed using the sequences of Windows Application Programming Interface (API) calls and their frequency of appearance. This is an effective method to not only classify malware from benign files, but also to identify malware variants within existing malware families. The similarity analysis technique related to data mining provides a consistent method to help in tracking possible groups as it can profile malware behavior and actions that may match previous profile attacks. This approach can help to recognize future attacks, improve malware detection tools, and develop better user education about the dangers of malicious software.

2. Background

Literature surveys on malware detection and malware variants show that there is no single technique that could detect all types of malware as most rely on syntactic properties rather than the semantics of malicious code (Alzarooni, 2012). However, generally there are two techniques commonly used for malware detection: *signature-based* detection and *anomaly-based* detection (Dinaburg et al., 2008; Lawton, 2002; Birrer et al., 2009). All of the Anti-Virus (AV) engines use malware signatures as an essential method to identify malicious content. The malware signature is a byte sequence that uniquely identifies a specific malware. Typically, a malware detector uses the signature to identify the malware by the byte sequence, which acts like a fingerprint of the malware program. Most AV programs are search engines supplied with a database containing information about existing or known malware, and this can be used to search for code signatures such as byte sequences while scanning the system. A malware detector scans the system for characteristic byte sequences or signatures that match with the one in the database and if found blocks the malware access to the system. The signature matching process is called signature-based detection and most traditional AV engines use this method. It is a very efficient and effective method to detect known malware (Venkatraman, 2009). However, a major drawback is the inability of this method to detect new and unknown malicious code as new signature generation involves manual processing and requires detailed code analysis. To overcome signature-based methods, obfuscation techniques such as polymorphic malware is used which has an in-built polymorphic program/engine that can generate new variants each time it is executed and, as well, a novel signature. Therefore, signature based approaches would fail to detect such malware. On the other hand, anomaly-based detection uses the knowledge of normal behavior patterns to decide if a program code is malicious. This approach has the ability to detect even some zero day attacks. However, it is very difficult to accurately specify the system or program's behavior and thus these approaches usually result in more false positives than signature based methods.

2.1. Obfuscation techniques

Malware coders are continually developing new techniques for transforming binary code that cannot be detected by AV scanners, and their level of sophistication is continuing to grow. Malware coders are applying obfuscating techniques (O'Kane et al., 2011) such as *packing* (Guo et al., 2008), *polymorphism* (Stepan, 2005), *Oligomorphic*, and *metamorphism* (Qinghua and Reeves, 2007) in order to transform existing malware code with signatures that are either disguised or are changed from those that might be held in a signature directory without affecting the original functionality or purpose. By looking at real world malware threats such as Parite, Rimecud, Alcan, Rbot, CeeInject, Nachi Bagle, and many other malware, it found that coders are recycling existing malware with different signatures by using obfuscation techniques instead of creating entirely new codes. This has created a serious threat for computer security. We define the term obfuscation to mean the modification of the program code in a way that preserves its functionally with the aim to reduce vulnerability of any kind to static/dynamic analysis and to deter reverse engineering by making the code difficult to understand and less readable. Code Obfuscation is an evasion method used to morph the program code in such a way as to make it hard to read and difficult to understand without changing the main goal of the program. Obfuscation techniques are used by malware coders as well as legitimate software developers. They both use code obfuscation techniques for different reasons. For instance, cybercriminal use it to evade antivirus scanners since it modifies the program code to produce offspring copies, which have the same functionality but with different byte sequence or 'malware signature' that is not recognized by antivirus scanners. But legitimate software developers use it for various reasons such as reducing file sizes, protecting against piracy, etc.

Malware coders are continually developing new techniques for creating and applying obfuscation techniques $T(P)$ on a malware program (p) to produce an obfuscated program (p'), thereby making it very difficult to reserve engineer and decipher the signature successfully, even though the two programs, p and p' have the same functionality and exhibit the same affect. On the other hand, since p and p' have different byte sequences, AV engines and reverse engineers are applying de-obfuscation techniques $D(p')$ on

the obfuscated program (p') in order to analyze the malware and to detect the malware as shown in Fig. 2.

Polymorphic (also known as *Code Encapsulation, and code packing*) has been in existence for more than 20 years. This method uses encryption and data appending/data pre-pending in order to change the body of the malware, and further, it changes decryption routines from infection to infection as long as the encryption keys change. This has led AV experts to develop different scanning techniques, from simple byte sequence matching to more complex techniques, which can be very difficult to detect (Beaucamps, 2008) such as the ones explained in Perriot and Ferrie (2004). According to SophosLabs (SophosLabs, 2014) 75% of new malware variants were developed using this new technique. The 2012 Symantec Intelligence Report (Symantec, 2012) described how detecting polymorphic malware is a hard task for traditional scanners to detect as it constantly changes its code. and the volume of polymorphic malware has increased dramatically. Polymorphic malware such as w32.Polip and w32.Detnat are much more difficult and complex to analyze than the other types of malware. Interestingly enough, the Enterprise Strategy Group (ESG) surveyed 315 security professionals working at North American-based enterprise organizations. 40% of surveyed security professionals claim they are not very familiar or not familiar at all with polymorphic malware (Oltsik, 2013).

Oligomorphic (also known as *Semi-Polymorphic*) is an encrypt which has a small finite number of different decryptors, which mutate its own decrypt routines. Whale Virus, which emerged in 1990, is an example of using this technique, and also Memoral. In both oligomorphic and polymorphic techniques, the code may change itself each time, but the semantic is not modified.

Metamorphic malware changes the code itself without the need of using encryption. In general, there are four techniques commonly used for metamorphic obfuscation. These are (i) *Dead-code Insertion* (Also known as *Garbage Insertion*) which does not do anything, such as NOP (No Operation Performed), (ii) *Code Transposition* that changes the instructions such as using JMPs instructions so that the order of instructions is different than the original one, (iii) *Register Reassignment* (also known as *Variable Renaming*) which replaces program identifiers like registers, labels and constant name. For example replacing PUSH EBX with PUSH EAX to exchange register names, and (iv) *Instruction Substitution* (also known as *Equivalent Code Replacement*) which replaces the instructions with different instructions that still have the same result. Some coders use a database dictionary of equivalent instruction sequences to make it easier and faster.

Packers are commonly used today for code obfuscation or compression, and there are literally thousands of discrete packing tools (Rogers, 2011) that are available for download. Packers are software programs that could be used to compress and encrypt the PE in secondary memory and to restore the original executable image when loaded into main memory. Malware coders do not need to change several lines of code to change the malware signature. For malware coders using packers is a common practice now for many reasons, such as: protecting their code by hindering the understanding of the code's purpose, running malware faster, and making malware easier to transform, thereby producing malware more resistant to static and dynamic analysis (Roundy and Miller, 2013). Most packers use a small unpacking loop to decompress advanced decompression algorithm that unpacks the actual payload (Roundy and Miller, 2013).

In a nutshell, malware coders' level of technical sophistication is continuing to grow. They are continually developing these new techniques to create malware that cannot be detected or that makes it difficult for AV engine detection. These experimental tests have shown that all of the above techniques can be used to fool current detection engines. Hence, in order to cater to all the above mentioned techniques, as a start this research focuses first on unpacking and extracting the behavioral features of malware code, rather than the typical "pattern matching" detection process that are evaded by obfuscations of the byte sequence through packing, metamorphic and polymorphic techniques.

2.2. Malware: old wine in a new bottle

As access to computers and the Internet has become widespread, cybercriminals have become increasingly sophisticated in their techniques of malware production. Cybercriminals who apply their skills for profit have increasingly supplanted the thrill-seeking, computer-savvy hackers of the 1970s and 1980s, who promoted an anarchist like culture of a 'free' Internet (Cao and Lu, 2011). The transition to increasingly organized forms of cybercrime is reminiscent of a generational change: the hackers of the 1970s and 1980s tended to act alone and were motivated by fun, while the skilled IT criminals of today specialize in one area and hire out their skills to criminal organizations. Protection of information from malicious coders (who are variously called 'black hats', 'hackers', and 'crackers') is of utmost importance in today's information society, which has high level of cybercrime (UNODC, 2013). There is also a risk of escalation if the tools become more secure from detection. Today the goals of those releasing binaries have moved from showing off skills in coding and fun, to financial gain (Broadhurst and Grabosky, 2005). More recently, it involves a range of illicit activities mainly oriented toward profit making and many malware codes are developed by nation state actors or their surrogates for strategic or tactical offensive action against 'enemies' rather than as crime-ware (e.g. Stuxnet, the worm created for Operation Olympic Games against Iran's nuclear facilities). The information security industry is another potential distribution vector for malware as penetration testing generates new codes capable of avoiding filtering and other malware detections. Such malware can be sold or made available by delinquent security professionals. Computer systems may be hacked using malware to steal the data they contain (e.g. banking and credit card details) and stolen data is sold to thriving underground markets. The Internet now offers the single largest platform for buyers and sellers of goods and services. As today's society has becomes more digital in terms of smart cards, electronic money, electronic checks, digital cash, stored value cards and online banking, more opportunities for threats to e-security have been created. This threat evolution has escalated to a great extent as society has moved online and the risks of theft from financial transactions and payments via identity theft have become more commonplace (Broadhurst and Chang, 2013). These threats faced by individuals and organizations continue to evolve as offenders aggressively develop techniques to steal money and financial credentials or personal information. The Internet is also a vehicle for fraud and deception, as in the case of spurious investment offers, marriage proposals, and other fraudulent overtures.

Cybercriminals continue to create new methods for developing and distributing malicious binaries. Crime toolkits like exploit kits and botnets are serious threats in malware production and can be used easily by offenders. There is also a risk of escalation if the tools become more secure from detection. Binaries attacks generated from these toolkits can deliver automated instructions to commit malicious activities, and have become increasingly organized and purposefully directed. Botnets, in particular, are a clear example of this trend. Botnet attacks such as *Slapper, Coreflood, Kraken, Sinit, Nugache, Rustock, Conficker, ZeuS, ICE X, SpyEye, Blackhole, NGR* and many others are considered among the most serious threats to Internet security. Botnets give their offenders (bot operators) the potential to remotely control a large number of computers that are compromised (zombies) that allow the zombie computers to be controlled remotely through established command and

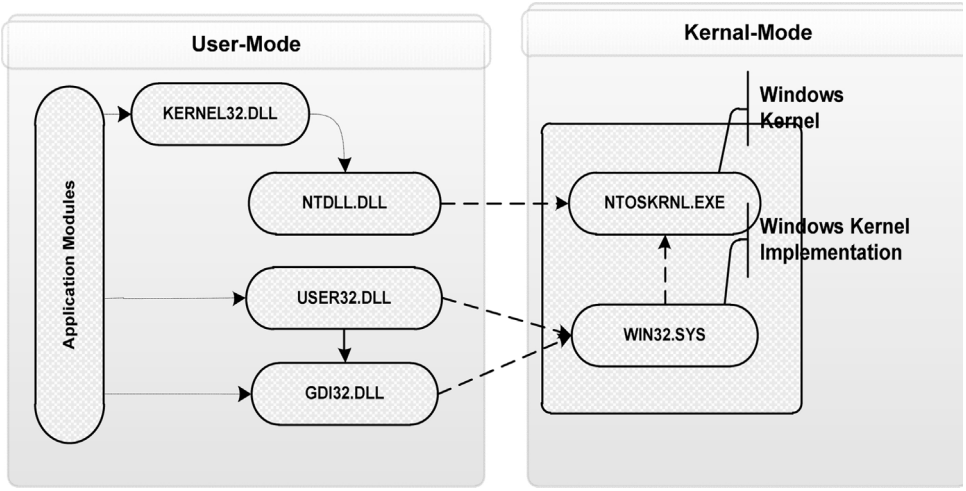

Fig. 1. API calls interface DLLs and their relation.

control channels (C&C). Collectively, under the control of C&C servers, botnets can become powerful and effective slave computing assets that can be rented for illegal activities. Such activities include phishing attacks, installing backdoors or rootkits on host systems to steal sensitive information (i.e. keyloggers that capture identity data, and passwords), send spam emails, and launch large scale Distributed Denial Of-Service (DDoS) attacks. The C&C channels are becoming more sophisticated and have developed from earlier misuse of Internet Relay Chat (IRC) to Hyper Text Transfer Protocol (HTTP), and changing the architecture from a centralized structure to a decentralized structure (i.e. P2P) and fast flux network services.

ZeuS Kit allows relatively unskilled offenders to create and manage a botnet capable of stealing a wide variety of information from victims' machines. Since its first appearance in 2007, it has grown into one of the most dominate families to steal banking and private credentials. Many versions of ZeuS have been thoroughly investigated by the security community because of it prevalence. In 2010, it was widely reported in the press that the full source code of the infamous crimeware toolkit, ZeuS, had been leaked onto various Internet sites. This has led to the development of several centralized trojans based on ZeuS, such as ICE IX, KINS, and the more successful Citadel. Also, decentralized trojans based on ZeuS appeared in September 2011 and peer-to-peer (P2P) known also as P2P ZeuS or *GameOver ZeuS*.[1] It is a significant improvement from all other versions of ZeuS because it replaces the centralized server with a P2P network (Andriesse et al., 2013). AV industry has continued to see large numbers of intrusions attributed to the ZeuS kit and, as well, the effect of the source code release has led to the breeding of numerous variants within the malware network.

3. Overview

Windows malicious software has come a long way. Lots of techniques and methods exhibit false alarms such as false positives as they do not perform sufficient statistical analysis or mining analysis to determine if the anomaly was 'actually' malicious (Jacob et al., 2008). Therefore, in this research, anomaly-based detection analysis is adopted to perform introspection of the program code with the goal of determining various dynamic properties of API function calls that are extracted from these codes in a controlled environment.

Understanding the format of program executables (PE) and API call are of critical importance for identifying the malicious codes. Hence, some background information pertaining to API calls and PE structure that is exploited by malwares are presented here.

3.1. Portable executable

Windows Portable Executable (PE) file format introduced by Microsoft is the standard executable format for all versions of the operating systems on all supported processors. Most programs in Windows are constructed by accessing the Windows API through functions available in dynamic link library (DLL) on the system. Microsoft provides a great number of DLLs, and each DLL can be used by more than one program at the same time.

The core of the Windows OS consists of the Application Programming Interface (API). An API is a set of commands, functions, and protocols which programmers can use when building software to interact with the operating system. API function calls reflect the behavior of executable codes and enables the program to exploit the power of OS – in this case the Windows OS. The Windows API function calls comprises of functional levels such as system services (base and advanced), graphical interface, user interface, network resources, windows shell and libraries, etc. Windows API refers to a set of APIs that provide access to different functional categories, such as networking, security, system services, and management. Since the API calls reflect the functional levels of a program, analysis of the API calls should lead to an understanding of the behavior of the file (Sharif et al., 2008).

This paper focuses on extraction and analysis of API call, and how to automate the entire detection process. In order to automate the inspection of these features in the PE, it is important to note that the PE structure and that the data structures on secondary memory (Hard Disk) are the same data structures that are used dynamically in main memory (RAM). For example, when the function LoadLibrary is called to load the executable into memory, a data structure such as the IMAGE_NT_HEADERS is created identical on disk and in the memory. The Windows loader is responsible for deciding what section the PE need to be mapped in. Hence, the mapping between the two memories is consistent, such as the higher offset in the disk file is similar in position to higher memory address when mapped into the main memory. The PE files have a number of sections and headers, header start with the signature of the PE, file properties, such as the number of sections and timestamp. The section table

[1] Uses a decentralized network infrastructure of compromised personal computers and web servers to execute command-and-control.

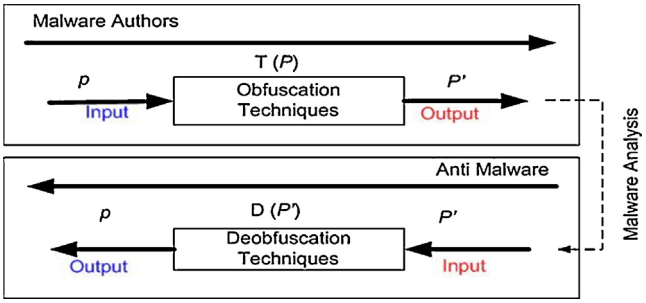

Fig. 2. Obfuscation transformation.

Table 1
DLLs and their main function.

DLL name	Main function
Kernel	File system and inter-process communication
Advapi	Registry access; service and driver management
User	Creating Windows and GUI elements
GDI	Drawing on the screen
Shell	Create shell objects

has a number of sections and the most important high-level sections are code sections (.text) and data sections (.data). The .text section is the default section for code and the .data section stores writable global variables and also contains the file's Original Entry Point (OEP), which refers to the execution entry point (where the file execution begins) of a portable executable file. The read-only data section (.rdata) is the read-only memory region, i.e. the string which is used in the program or global constant arrays.

In the Windows Operating System (OS), user applications rely on the interface provided within a set of libraries, such as Kernel, GDL, NTDLL and User in order to access system resources including files, processes, network information and the registry. This interface is known as the Windows API. Applications may also call functions in NTDLL.DLL known as the Native API. The Native API function performs system calls in order to have the kernel provide to the requested service. Malicious codes are able to disguise their behavior by using API functions provided under Windows environment to implement their tasks. Therefore, the focus of this research is to identify all Windows API call features to understand their behavior. NTDLL serves as the fundamental user-mode interface of the Windows Native APIs. Fig. 1 shows the relationship between the DLLs. Table 1 shows some DLLs and their main function. NTDLL contains the Native APIs, while KERNEL, USER, and GDI consist of the main Win32 Core Subsystem APIs. Native API is the set of functions exported from both NTDLL and NToskrnl.exe. The actual implementations of the Native API calls reside in NToskrnl.exe, which means that Native API is the most direct interface into the Windows kernel.

3.2. Need for the study

In previous work (Alazab et al., 2011), I proposed and evaluated a method of employing several data mining techniques to detect and classify new malware based on the frequency of appearance in Windows API calls. We have employed several robust classifiers, namely Naïve Bayes (NB) Algorithm, k-Nearest Neighbor (kNN) Algorithm, Sequential Minimal Optimization (SMO) Algorithm with 4 differents kernels (SMO – Normalized PolyKernel, SMO – PolyKernel, SMO – Puk, and SMO – Radial Basis Function (RBF)), Backpropagation Neural Networks Algorithm, and J48 decision tree. Result show that the frequency of Windows API calls can be used effectively to detect new malware. Therefore, the focus of this research is not only to detect malware but also to identify variants based on the API system call sequences and their frequency of appearance.

The prime motivations for this research are: (1) A recent malware trend is to fool detection engines by applying obfuscation techniques on known malware (Sharif et al., 2008), (2) identifying benign files as malware (false positive), and malware as benign (false negative) are becoming more common, (3) rates of failure to detect obfuscated malware is high (Symantec, 2011; Symantec Enterprise Security, 2009, 2010), (4) the current detection rate is decreasing, (5) current malware detectors are unable to detect zero day attacks (Symantec Enterprise Security, 1997; RSA, 2011), (6) difficulties around predicting malicious attacks and variants, and (7) from other study on related work (Venkatraman, 2009; Perriot and Ferrie, 2004; Bruschi et al., 2006; MetaPHOR, 2010; Ferrie and Szor, 2003; Linn and Debray, 2003; Chang and Atallah, 2002) it has been found that the statistical modeling of hidden malcode that predominantly uses Windows API calling sequence for evading detection is yet to be explored. Hence, the main focus is to address zero day attacks that use code obfuscation, which poses a challenge for digital forensic examiners (Alazab et al., 2009) with the limitations of signature based detection (Tang et al., 2010; Santos et al., 2013b).

Extracting features from the obfuscated executables for reverse obfuscation is labor intensive and requires deep understanding of kernel and assembly programming (Alazab et al., 2010a, b). Recent studies (Bilar, 2007a; Moskovitch et al., 2008; Bilar, 2007b; Rad and Masrom, 2010, 2011; Santos et al., 2010) have used analysis of Operation Code (OPcode) for generation to birthmark the portable executable files. Use of statistical analysis of file binary content including statistical N-gram modeling techniques (Santos et al., 2013b; Stolfo et al., 2005; Wang et al., 2009) have been tested in identifying malcode in document files, and does not have sufficient resolution to represent all class of file types. Based on other studies (Venkatraman, 2009; Linn and Debray, 2003) it has been found that the statistical modeling of malware code that mainly uses obfuscation engines to produce morphed copies of an original program for evading detection is yet to be explored. Positive contribution in understanding malware behavior and detecting malware variants using API sequences and their frequency of appearance is, therefore, essential.

4. Related studies

In recent years machine learning and data mining have been the focus of many malware researchers and analysts to counter the challenge of obfuscation techniques and malware detection. Data mining is also referred to as 'knowledge discovery' in databases. Frawley et al. (1992) defines it as "*The nontrivial extraction of implicit, previously unknown, and potentially useful information from data*". It is also defined as "*The science of extracting useful information from large data sets or databases*" (Hand et al., 2001). To our knowledge, Schultz et al. in 2001 were the first to apply data mining to the detection of different malicious programs based on their respective binary codes (program headers, strings and byte sequence) on several classifiers. A few years later, in 2004, Kolter and Maloof (2004) tested the performance of various classifiers such as Naïve Bayes, Support Vector Machine (SVM) and plotting ROC curves using decision tree methods, and improved the results by using n-grams of byte codes as features. In 2004, Sung et al. (2004) and Xu et al. (2004) proposed a method for computing the similarity between executables of API call sequences made in an attempt to detect polymorphic and metamorphic variants of Malware. Authors defined signature as an API sequence of calls and started the reverse engineering process from decompressed 16 binaries. They were then passed through a PE file parser, then extracted and mapped to the sequence of Windows API calls, and lastly passed through the similarity measure module, where

similarity measures such as, Euclidian distance, Sequence alignment, Cosine measure, extended Jaccard measure, and the Pearson correlation measure were used. Although these similarity measures enable Static Analyzer of Vicious Executables (SAVE) to detect polymorphic and metamorphic malwares efficiently against 8 malware scanners, the main limitation of this method is that the frequency appearance of the calls are not considered, however, this would add another detection layer to the overall system. In 2005, Malan and Smith (2005) added a temporal consistency element to the system call frequency and calculated the frequency of API system call, but it can be argued that the study was not representative as it only used nine variants of worms. In 2006, Kolter and Maloof (2006) described the use of machine learning and data mining to detect and classify malicious executables. Kolter tested several classifiers including IBk, naive Bayes, SVM, decision trees, boosted naive Bayes, boosted SVM, and boosted decision trees. Kolter found that support vector machine performed exceptionally well and fast compared to the other classifiers. However, this work did not focus on measuring malware similarity.

Another anomaly based detection attempt was to identify polymorphic malware based on the analysis of Windows API execution sequences called Intelligent Malware Detection System (IMDS) by Ye et al. (2007). IMDS was developed using the Objective-Oriented Association (OOA) mining based classification system and a large data set was used for the experiment. For detection purpose, a Classification Based on Association rules (CBA) technique such as Naïve Bayes, SVM and Decision Tree were used. The result was compared against anti-virus software programs such as Norton, Kaspersky, McAfee, and Dr.Web. In 2010, the authors of IMDS had incorporated the CIDCPF method into their existing IMDS system with larger dataset, and called it CIMDS system (Yanfang et al., 2010). CIDCPF adapted the post processing techniques as follows: first Chi-square testing was applied and insignificant rule pruning, followed by using database coverage based on the Chi-square measure rule ranking mechanism and Pessimistic error estimation, and finally prediction was performed by selecting the best first rule. Their results were good, but the accuracy was low at approximately 88%.

API calls based features are not only good in classifying malware from benign, but also in detecting injected malicious executables. In the article 'Detection of Injected, Dynamically Generated, and Obfuscated Malicious Code' (DOME) (Rabek et al., 2003), the authors used static analysis to identify the locations of system calls within the software executables, and monitored the executables at runtime to verify the location. However, DOME is unable to deal with the many obfuscated malware or malware variants. In 2010, Shankarapani et al. (2010) showed that the frequency of Windows API calls can be a good feature to classify malware. Authors have performed a static analysis to measure the similarity of malware and benign files. Authors used Similarity analysis to compute the mean value for 3 similarity measures (Cosine similarity, Extended Jaccard measure, and Pearson correlation). Also, SVM kernel RBF was used to classify malware. However, the work was based only on term frequency of Windows API and did not include it sequence of occurrence. Another drawback of the result was that the Receiver Operating Characteristic (ROC) curve was not of a high rate compared to earlier studies. In the same year of 2010, Cesare and Xiang (2010) employed both dynamic and static analysis to classify malware in an attempt to identify malware variants using string edit distances based on the control flow graph matching algorithm as a signature. Authors enhanced their work (Cesare et al., 2014) by using approximate matching of control flow-graphs, and extracted q-grams and k-sub-graphs of sets of control flow-graphs to create feature vectors. From these feature vectors authors were able to construct distance metric and similarity search. However, these works are entirely based on syntactic features of control flow graphs rather than using any semantics features such as API calls. Most of the previous work relating to similarity based detection exhibit some drawbacks. They performed their experiments either using small datasets or small set of API calls or PE sequences, and more importantly many did not give importance to new malware due to obfuscation techniques adopted in existing malware families.

Table 2
Dataset of malware and benign.

Type	Qty	Max. size (KB)	Min. size (KB)	Avg. size (KB)
Benign	15,480	109,850	0.8	32,039
Virus	17,509	546	1.9	142
Worm	10,403	13,688	1.6	860
Rootkit	270	570	2.8	380
Backdoor	6689	1299	2.4	685
Constructor	1039	77,662	0.9	1193
Exploit	1207	22,746	0.5	375
Flooder	905	16,709	1	1397
Trojan	13,201	17,810	0.7	1819

5. Datasets

The database used in this experiment consists of 66,703 executable files in total of which 51,223 are malware, with the remaining being benign datasets as shown in Table 2. Such large malware datasets with obfuscated and unknown malware used in this research study have been collected from the Honeynet project, Heavens (2011) and other sources. The 15,480 benign datasets include: application software such as databases, educational software, mathematical software, image editing, spreadsheet, word processing, decision making software, internet browser, email and many others system software, programming language software and many other applications. Both malware/benign files have been uniquely named according to their MD5 hash value.

6. System overview

In order to detect both malware variants and distinguish benign executables, this proposed methodology mainly focuses on extracting the API sequences and their frequency of appearance. Similarity based detection methods were then used for identifying malware variants and classifying them within existing malware families. The method used is semi-similar to the one suggested and used by Santos et al. (2010, 2013a), but instead of using OPcode as a distinguishing feature, this research used API calls features for its importance as mentioned in the above sections. Fig. 3 gives an overview of the proposed system that uses the similarity based detection methodology partially adopted from an earlier research paper (Alazab et al., 2010b). The proposed framework consists of two essential steps.

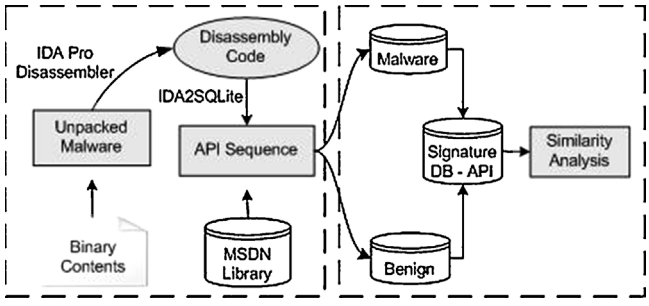

Fig. 3. System overview.

6.1. Step 1: unpack and disassembly binaries

Both open source and commercial tools have been used to de-obfuscate and unpack the dataset samples, and each of these tools uses different de-obfuscation techniques to each other. Packing has become the most common method for malware coders. Understanding the unpacking process and how it restores the original binary in memory enables analysts to unpack payloads. Generally, the unpacking stub performs three tasks:

- To unpack the original executable into memory.
- To resolve the imports from the original executable.
- To transfer execution to the original entry point.

Enormous numbers of malware samples are propagated daily of which many are obfuscated. Malware analysts cannot rely on static analysis because of its limitations. As a result, analysts have proposed to build semi-automated or automated dynamic solutions either statistically like the one implemented in Ether (Dinaburg et al., 2008), and Eureka (Sharif et al., 2008), or dynamically such as PolyUnpack (Royal et al., 2006), Renovo (Kang et al., 2007), and OmniUnpack (Martignoni et al., 2007). These solutions involve monitoring the usage of memory and identifying unpacked code generated at runtime. Here I provide some description and comments about each one of them: PolyUnpack is an automated unpacking technique to extract the hidden code through process execution by using the Windows debugging API, which works by comparing any executed code against the executable's static code model. The second tool, Renovo, is implemented using the QEMU emulator and supports multiple layers of unpacking to automatically extracts the hidden code of a packed executable and provides valuable information on finding the Original Entry Point (OEP). PolyUnpack and Renovo methods traces the execution until the unpacking routine terminates. In contrast, OmniUnpack reverses the single packed binary and runs an anti-virus scanner on the recovered code only, and once the behavior of the packer binary is observed by the system an anti-virus scanner is then used to check whether the newly generated code contains any known malicious patterns. Eureka is similar to OmniUnpack except that Eureka tracks execution at the system call level. Eureka uses statistical bigram analysis and coarse-grained execution tracing method and provides several Windows API resolution techniques that identify API calls in the unpacked code. Lastly, Ether, is based on an application of hardware virtualization extension such as Intel VT, and resides outside the operating system.

All data set files collected in this experiment were preprocessed for anomaly testing. In order to translate a program into an equivalent high-level-language program based on the binary content, the most reliable disassembly tool, namely, Interactive Disassembler Pro (IDA Pro) (IDA, 2011) since it can disassemble all types of non-executable and executable files (such as ELF, EXE, PE, etc.), was used. Also, the IDA Pro was selected because it is a component of the automation process that automatically recognizes API calls for various compilers and can be further extended with Python programs and compiled plugins, resulting in incredibly powerful implementation with flexible levels of analysis and control. IDA Pro is a recursive traversal disassembler, which deploys several heuristics and library signatures to resolve imported kernel calls. IDA Pro loads the selected file into memory to analyze the relevant program portion to create an IDA database whose components are stored in four files: .id0 that contains the content of a B-tree-style database, .id1 that contains flags describing each program byte, .nam that contains index information related to program locations, and .til that is used to store information concerning local type definitions to a given database. IDA Pro generates the IDA database files into a single IDB file (.idb) by disassembling and analysing the binary of the file. The fully-automated system proposed uses Python programming language, which generates .idb automatically from the set of malware samples.

Table 3
The top ten packers classes in our dataset.

Packer name	Percentage (%)
UPX	17
Petite	15
FSG	12
PECompact	7
PELock	5
Upack	4
NsPack	3
AsProtecect	3
Armadillo	3
MEW	2

IDA Pro shows if the program might be packed or unpacked, and I, therefore, separated the packed and unpacked executables. If the executable is labelled as unpacked, it is necessary to go to step 2 to extract API calls as explained in the next step, but if an executable is classified as packed, the IDA Pro Universal PE Unpacker plug-in is used in an automated way to unpack the binaries, see (DataRescue, 2005) for details, also used Ether tool for unpacking details (Dinaburg et al., 2008), If the plugins fails to unpack the program, I used a IDASQLit plugin developed by the author (detailed in step 2), which simply applies pattern recognition techniques that have similar tactics to OmniUnpack and Eureka, which, after reversing the single packed binary, run an anti-virus scanner (e.g. ClamAV (Clam)) on the recovered code. Once the behavior of the packed binary is observed by the system, an anti-virus scanner then is used to check whether the newly generated code contains any known packer patterns. Within the sample size, it found that more than 100 different Packers/Crypters were used. Table 3 shows the top packers names were used in the dataset. Results shows that more than 80% of the sample were packed or encrypted, which is similar to the findings of (Thrilling, 2008; Bruschi et al., 2006; Martignoni et al., 2007).

6.2. Step 2: extract API calls

IDA Pro provides access to its internal resources via an API that allows analysts to create plug-ins to be executed by IDA Pro. This research used SQLite (2011), which is a software library that implements a self-contained transactional SQL database engine. The Python program automatically runs and creates the plugin to use SQLite with IDA Pro for generating the database (.db). An interface for accessing the database file (.db) was then developed so that the results from the assembly code of the malware stored in the database could be used for better binary analysis.

The SQLite plugin used, generated eight tables (Blocks, Functions, Instructions, Names, Maps, Stacks, Segments, TargetBinaries), each of them containing different information about the binary content. The Function table contains all the recognizable API system calls and non-recognizable function names, and the length (start and the end location of each function). Instructions table contains all the operation code (OP) and their addresses and block addresses. The Map table contains the function address and source of block address and the destination of the function address. The Name table contains function addresses, the name of the function, and the type of the function. The Stacks table contains function address, the stack name, and the start and the end address. The Segments table contains information that describes each segment in an executable file, segment name (Code, Data, BSS, _idata, _tls, _rdata, _reloc, and _rsrc), and the segment length. Finally, TargetBinaries contain the file name, path name, MD5, and start and the end of analysis.

For the analysis of malware behavior, frequency of API calls and the call sequence were considered. As a reference for this step (Jianyun Xu et al., 2007), the API calling sequence extraction performed by; (i) locating the Imported Address Table (IAT), (ii) generating binary lookup tree from all imported API, (iii) extracting the CALL and JUMP instructions and their target addresses, (iv) looking up the target address in the binary lookup tree to find the corresponding API. (vi) Lastly, mapping the API name with its module name to a 32 bits unique global API id by a lookup table. The Windows application programming interface (API) from The Microsoft Developer Network (MSDN) (2011) was downloaded. A fully automated system using Python programming language that integrates with IDA Pro and SQLite was implemented to enable the processes required to compare and match the API from MSDN and the API calls generated in the database (.db) for the malware sample set. In addition, all the API calls that are associated with malware code were listed.

7. Similarity measures

Similarity mining is a detection method based on the analysis of the similarities in distance measures. The most direct way to measure the similarity between two features is to compute the distance between the new class and every other case in the knowledge base. Similarity based mining has been implemented to identity malware variants. In this section, an approach for measuring similarities between executables is detailed. By using the mechanism described in the system overview system, we construct our database signature. A signature is sequences of API calls and their frequency of appearance that extracted from the database sample. The signature database has been used to statistically calculate and compute the similarity measures. Five distance measures have been adopted to analyze and compute distances between malware variants and benign executables from various families. The signature database has been generated from the existing datasets to produce fingerprint or birthmark for each record based on the API calls that the executable programs had used as shown in Table 4.

Similarity based detection is well-suited for malware analysis either static or dynamic analysis, where firstly the executable program is disassembled using reverse engineering tools. Each disassembled executable (P) represents a vector of functions x, y. (P') is the variant malware of the original executable (P). Each function is represented as an array of vector of functions. The similarity between the functions of a program (P) and (P') is computed. These measures are referred to as similarity coefficients, and have values between 0 and 1. A value of 0 indicates that the two vectors are completely similar, while a value of 1 indicates that the vectors are not at all similar. The value is then compared with the threshold value, the threshold for similarity (t) was chosen as 0.6, the threshold was chosen for empirical results in Cesare and Xiang (2010) and Cesare et al. (2014). For example, if the similarity exceeds a given threshold for any malware in the database, then the program under inspection is considered a variant of that malware, and therefore malicious.

In this section similarity analysis has been performed. In short, the maliciousness of a code is estimated using these measures. For instance, malware such as Win32.Evol (Orr, 2006) has a multiple variants of the same sample malware because of the obfuscation methods adopted by the malware coders. Similarity based detection approaches can be used among the variants to check whether the variant is the child of the sample under inspection with similar features.

In many areas of studies such as image retrieval, the Euclidean distance is widely used. But it may not be suitable for all applications like in our malware analysis, therefore for this research we have used five different traditional similarity functions namely; Canberra, Chebyshev, Manhattan, Correlation, and distances which are the most popular measures of similarity for vector, details are below. u and v vectors are arbitrary samples, the distance computation used to determine whether they are malware or benign:

u, v The sequences of Windows API sequence calls and frequency of samples (malware or benign)
n is the dimensional vector.
D is the distance between vectors u and v.

A. Canberra distance

$$D = \frac{\sum_i |u_i - v_i|}{\sum_i |u_i + v_i|} \tag{1}$$

B. Chebyshev distance

$$D = \mathrm{MAX}_i |u_i - v_i| \tag{2}$$

C. Manhattan distance

$$D = \sum_i |u_i - v_i| \tag{3}$$

D. Correlation distance

$$D = 1 - \mathrm{frac}(u - \bar{u})(v - \bar{v})^T ||(u - \bar{u})||_2 ||v - \bar{v}||_2^T \tag{4}$$

where $\bar{u}$ is the mean of a vectors elements and n is the common dimensionality of u and v.

E. Euclidean distance

$$D = ||u - v||_2 \tag{5}$$

8. Experiments and results

The experimental investigation analysis of the similarity and classifications was carried out by implementing the various distance measures in Python Programming Language. The experiment was run in Intel Core i5-2520 M CPU @ 2.50 GHz, 8.00 GB of RAM with Windows 7 professional as the operating system. The implemented analysis in Python aided in the programs classification and detecting malware variants. The analysis evaluated using very large real-life malware datasets. The datasets were obtained through public databases explained in Section 5.

The value ranges from 0 to 1, where 1 means that the two executables are completely different and 0 means that the executables are identical. The distance system was able to automatically classify malware samples from the benign samples also identify high similarities in the most of the malware variants. For the KNN in the next section Manhattan distance has been chosen because of its efficiency and robust over other distance measures, as it shows in Table 8. Manhattan distance show high robust performance in the domain of malware (Cesare and Xiang, 2010; Cesare et al., 2014).

Similarity analyses for two malware families (Delf and Dadobra) are shown in Tables 5 and 6. The highlighted cells identifying a malware variant is defined as having a distance less than or equal to 0.6. As shown the entire cell in the Table 5(a) can be detected as a variant of the original malware Win32.Dadobra.

The results show from Tables 5 and 6 that there is low distance/high similarity between malware variants but not with the benign programs as shown in Table 7. The mean value of the above distance measures was calculated. A similarity report is generated by calculating the value for each signature in the signature database, and index highest similarity index indicates the most likely distinct variant or family. By comparing this index value with a threshold, decision is made whether the unknown executable

Table 4
Database signature of API calls.

ID	Called API	Class
Prg.1	$\sum_{i=0}^{5} API_1, \sum_{i=0}^{1} API_3, \sum_{i=0}^{10} API_{16}$	Malware
Prg.2	$\sum_{i=0}^{7} API_3, \sum_{i=0}^{2} API_{11}$	Malware
Prg.3	$\sum_{i=0}^{3} API_{22}, \sum_{i=0}^{4} API_1, \sum_{i=0}^{1} API_{20}, \sum_{i=0}^{2} API_6, \sum_{i=0}^{5} API_4, \sum_{i=0}^{2} API_3, \sum_{i=0}^{5} API_8$	Malware
Prg.5	$\sum_{i=0}^{2} API_{11}, \sum_{i=0}^{2} API_4, \sum_{i=0}^{7} API_6, \sum_{i=0}^{16} API_{10}$	Benign
Prg.6	$\sum_{i=0}^{5} API_4$	Benign
...	...	...

Table 5
The similarity matrix for two malware families.

(a) The Trojan Downloader Win32 Dadobra family

	.aa	.aj	.ak	.al	.am	.bf	.bh	.bw
aa	0.00	0.10	0.21	0.10	0.14	0.14	0.14	0.14
.aj	0.10	0.00	0.23	0.00	0.23	0.23	0.23	0.23
.ak	0.21	0.23	0.00	0.23	0.21	0.21	0.21	0.28
.al	0.10	0.00	0.23	0.00	0.23	0.23	0.23	0.23
.am	0.14	0.23	0.21	0.23	0.00	0.00	0.00	0.21
.bf	0.14	0.23	0.21	0.23	0.00	0.00	0.00	0.21
.bh	0.14	0.23	0.21	0.23	0.00	0.00	0.00	0.21
.bw	0.14	0.23	0.28	0.23	0.21	0.21	0.21	0.00

(b) The Worm Win32 Delf family

	.am	.d	.h	.m	.r	.t	.v	.z
.am	0.00	0.23	1.00	0.19	1.00	1.00	0.17	0.17
.d	0.23	0.00	1.00	0.07	1.00	1.00	0.10	0.10
.h	1.00	1.00	0.00	1.00	1.00	1.00	1.00	1.00
.m	0.19	0.07	1.00	0.00	1.00	1.00	0.10	0.10
.r	1.00	1.00	1.00	1.00	0.00	1.00	1.00	1.00
.t	1.00	1.00	1.00	1.00	1.00	0.00	1.00	1.00
.v	0.17	0.10	1.00	0.10	1.00	1.00	0.00	0.00
.z	0.17	0.10	1.00	0.10	1.00	1.00	0.00	0.00

Similarity analyses for two malware families (Delf (a) and Dadobra (b)) are shown in Table and across each other in Table 6.
The similarity problem between binaries is to determine if binary p is a copy or derivative of binary q, it is based on the definition detailed in Tamada et al. (2004).

Table 6
The similarity matrix for non-similar malware families.

	.aa	.aj	.ak	.al	.am	.bf	.bh	.bw
.am	0.10	0.19	0.29	0.19	0.17	0.17	0.17	0.20
.d	0.20	0.22	0.20	0.22	0.10	0.10	0.10	0.27
.f	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
.g	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
.h	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
.m	0.17	0.19	0.23	0.19	0.10	0.10	0.10	0.23
.r	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
.t	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
.v	0.14	0.23	0.21	0.23	0.00	0.00	0.00	0.21
.w	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
.z	0.14	0.23	0.21	0.23	0.00	0.00	0.00	0.21

Similarity analyses for two malware families (Delf Table 5 (a) and Dadobra Table 5 (b)) are shown in Table 5 and across each other in Table.
The similarity problem between binaries is to determine if binary p is a copy or derivative of binary q, it is based on the definition detailed in Tamada et al. (2004).

Table 7
The similarity matrix for non-similar benign.

	bintext	Dic	ffind	msgr	msimn	Msmsgs	Skype
bintext	0.00	1.00	1.00	1.00	1.00	1.00	1.00
Dic	1.00	0.00	1.00	1.00	1.00	0.98	1.00
ffind	1.00	1.00	0.00	1.00	1.00	1.00	1.00
msgr	1.00	1.00	1.00	0.00	1.00	11.00	1.00
msimn	1.00	1.00	1.00	1.00	0.00	1.00	1.00
msmsgs	1.00	0.98	1.00	1.00	1.00	0.00	0.98
Skype	1.00	1.00	1.00	1.00	1.00	0.98	0.00

Similarity analyses for two malware families (Delf Table 5 (a) and Dadobra Table 5 (b)) are shown in Table 5 and across each other in Table 6.
The similarity problem between binaries is to determine if binary p is a copy or derivative of binary q, it is based on the definition detailed in Tamada et al. (2004).

Table 8
Mean similarity matrix ($n \times m$).

Distance method	Malware to benign	Malware to malware	Benign to benign
Canberra	0.831	0.761	0.862
Chebyshev	61.27	31.45	79.98
Manhattan	141.32	96.24	180.03
Correlation	0.341	0.243	0.403
Euclidean	78.933	44.294	106.396

variant or not. Table 8 shows the mean values for 5 different similarity measurements applied to the entire dataset, when the threshold for the similarity ratio is less than or equal to 0.6.

It can be seen that there is high similarity between malware variants within the same family. Further, the experiments confirm that there is no similarity among the benign files, as it should be and make sense. Another important observation is low similarity between Malware dataset and benign dataset. This validate that the proposed system is able to clearly distinguish between malware and benign datasets. In conclusion, Malware can be classified using such distance and further a similarly test can be applied to detect malware variants. The proposed similarity based detection and classification of malware is further improved in performance by using feature selection, especially when large datasets require more computing memory and time for processing the extracted features. We used the distance functions in k-NN (next section).

9. k-Nearest Neighbor (kNN) algorithm

kNN is simple supervised machine learning algorithm. In this section we used the kNN for classifying malware and benign based

Table 9
validation for each class.

TP rate	FP rate	ROC area	CLASS
0.938	0.041	0.938	Malware
0.959	0.062	0.959	Benign
0.948	**0.051**	**0.966**	**Weighted Avg.**

on closest training instants in the Windows API calls. kNN has been used in many applications in data mining, statistical pattern recognition and many others. The object is classified based on a majority vote of its k nearest neighbors to the object. KNN has been used to predict whether an unknown executable is malicious or benign. In our experiments, the K-nearest neighbors are compute as follows with K:

- Store all training samples x_i^j in memory.
- Determine the parameter K = number of nearest neighbors beforehand. (A good k can be selected using cross-validation for example).
- Measure the distance between the query-instance (x) and all the training samples x_i^j. (using the Manhattan distance).
- Find the K-minimum distance between the query-instance (x) and each $K\, j^1_{\min}, j^2_{\min}, \ldots, j^k_{\min}$.
- Get all categories of training data for the sorted value under K.
- Find the weighted distance of the query-instance (x) from each of the k nearest points as follows:

$$w = 1 - \frac{\mathrm{dis}(x, x_i^j)}{\sum_k^{i=0} \mathrm{dis}(x, x_i^j)} \tag{7}$$

A standard grid search with 10-fold cross validation was performed to determine the best parameters for each classifier. The performance of the classifier was measured using the Receiver Operating Characteristic (ROC) curve and the overall accuracy. In ROC curve, the true positive rate is plotted in function of the FP rate for different points. Each point on the ROC curve represents a sensitivity pair corresponding to a particular decision threshold. A test with perfect discrimination has a ROC curve that passes through the upper left corner (100% sensitivity). Therefore the closer the ROC curve is to the upper left corner, the higher the overall accuracy of the test. Usually, ROC area higher (closer) to 1 is considered good, and closer to 0.0 is considered poor. The overall accuracy scores give the percentage of malware that are correctly classified as malware or not. These measures are defined from four different scenarios from malware or benign: True Positive (TP): Number of correctly identified malicious code, False Positive (FP): Number of wrongly identified benign code, when a detector identifies benign file as a malware. True Negative (TN): Number of correctly identified benign code. False Negative (FN): Number of wrongly identified malicious code, when a detector fails to detect the malware because the virus is new and no signature available yet. True detection Rate (TP rate): Percentage of correctly identified malicious code. TP Rate = TP/(TP + FN). False alarm Rate (FP rate): Percentage of wrongly identified benign code, given by: FP Rate = FP/(FP + TN) Overall Accuracy: Percentage of correctly identified code, given by: Overall Accuracy = (TP + TN)/(TP + TN + FP + FN). Table 9 shows the TP Rate, FP Rate, and ROC area for each dataset; Malware and benign.

10. Discussion and conclusion

Malicious software is one of the major security threats confronting computer systems and networks, which has been in existence from the early days of the Internet. In order to combat this constant growing threat, there is an urgent need to facilitate new malware forensic techniques to automate, identify, and characterize malware and variants. Malware analysis has evolved in sophistication of analysis tools such as automated techniques and machine learning tools. Unfortunately it becomes expensive and labor consuming to profile and detect. As analysis of malware continues to develop, it is apparent that the security industry is duplicating work in dealing with malware because of their focus on detection only. Efforts to address malware will be strengthened by looking beyond detection by understanding the purpose of the code, the *modus operandi*, and finding distinctive features of the code or 'fingerprints' for future profiling.

The technological skills of cybercriminals on malware code will certainly extend into the foreseeable future. As cyberspace grows, different opportunities will open up for organized crime groups. Profiling is not a new method, and the concept has been deployed in fighting crimes for a long time. Malware profiling is relatively new as an official method to analyze crimes. The detail analysis gains from malware profiling allows a clearer understanding of malware instances, behavior and variants, thus, laying the foundation for implementing countermeasures in a timely and appropriate manner. However, it is difficult to build the right profile for each and every malware code, simply because malware code can be programmed by someone, adjusted by another, and used by another. The motives and the circumstances should always be considered when a profile of malware code is assembled.

API calls reflect the behavior of a particular piece of code and, therefore, in this research work, the behavioral and structural features are automatically extracted from the binary of a program. The extracted calls were subjected to similarity based mining and machine learning methods to profile and classify the program files as either malicious or benign. The experimental investigations of the similarity analysis on large datasets of malware using the sequences of API calls and their frequency of appearance executables conclude that this method is effective in profiling malware. The empirical analysis of the experimental results yields that this proof-of-concept is effective and efficient in classifying malware. The results have achieved high true positive rates (0.94) and low false positive (0.051) rates, with an area under the ROC curve of 0.966, indicating that the method has been successful in detecting unknown malware.

Acknowledgments

The author wish the thank Prof Roderic Broadhurst, Dr. Robert Layton, Kate Macfarlane, Prof Peter Grabosky and the anonymous referees for their feedback on earlier drafts of this article. I acknowledge the assistance of the ANU Cybercrime Observatory (http://cybercrime.anu.edu.au).

References

Alazab, M., Venkatraman, S., 2013. Detecting malicious behavior using supervised learning algorithms of the function calls. Int. J. Electron. Secur. Digit. Forensics 5, 90–109.

Alazab, M., Venkataraman, S., Watters, P., 2009. Effective digital forensic analysis of the NTFS disk image. Ubiquitous Comput. Commun. J. 4, 551–558.

Alazab, M., Layton, R., Venkataraman, S., Watters, P., 2010a. Malware detection based on structural and behavioral features of API calls. In: The 1st International Cyber Resilience Conference, Perth, WA, pp. 1–10.

Alazab, M., Venkataraman, S., Watters, P., 2010b. Towards understanding malware behavior by the extraction of API calls. In: Cybercrime and Trustworthy Computing Workshop, Ballarat, VIC, pp. 52–59.

Alazab, M., Venkatraman, S., Watters, P., Alazab, M., 2011. Zero-day malware detection based on supervised learning algorithms of API call signatures. In: Vamplew, P., Stranieri, A., Ong, K.-L., Christen, P., Kennedy, P. (Eds.), The 9th Australasian Data Mining Conference. vol. CRPIT, 121. Australian Computer Society, Ballarat, VIC, pp. 171–182.

Alazab, M., Layton, R., Broadhurst, R., Bouhours, B., 2013. Malicious spam emails developments and authorship attribution. In: The Fourth Cybercrime and Trustworthy Computing Workshop (CTC), Sydney, NSW, pp. 58–68.

Alzarooni, K., 2012. Malware Variant Detection, Doctor of Philosophy. Department of Computer Science, University College London, London.

Anderson, R., Barton, C., Böhme, R., Clayton, R., Eeten, M., Levi, M., Moore, T., Savage, S., 2013. Measuring the cost of cybercrime. In: Böhme, R. (Ed.), The Economics of Information Security and Privacy, vol. IV. Springer, Berlin/Heidelberg, pp. 265–300.

Andriesse, D., Rossow, C., Stone-Gross, B., Plohmann, D., Bos, H., 2013. Highly resilient peer-to-peer botnets are here: an analysis of Gameover Zeus. In: 2013 8th International Conference on Malicious and Unwanted Software: The Americas (MALWARE), Fajardo, PR, pp. 116–123.

Beaucamps, P., 2008. Advanced metamorphic techniques in computer viruses. In: International Conference on Computer, Electrical, and Systems Science, and Engineering.

Bilar, D., 2007a. Opcodes as predictor for malware. Int. J. Electron. Secur. Digit. Forensic 1, 156–168.

Bilar, D., 2007b. Callgraph properties of executables and generative mechanisms. AI Commun. Netw. Anal. Nat. Sci. Eng. 20, 231–243.

Birrer, B.D., Raines, R.A., Baldwin, R.O., Oxley, M.E., Rogers, S.K., 2009. Using qualia and hierarchical models in malware detection. J. Inf. Assur. Secur. (Special issue on Detection and Prevention of Attacks and Malware) 4 (3), 247–255.

Broadhurst, R., Chang, L., 2013. Cybercrime in Asia: trends and challenges. In: Liu, J., Hebenton, B., Jou, S. (Eds.), Handbook of Asian Criminology. Springer, New York, pp. 49–63.

Broadhurst, R., Grabosky, P., 2005. Cyber-Crime. The Challenge in Asia. In: The Future of Cyber-Crime in Asia. Hong Kong University Press, pp. 347–360, http://www.jstor.org/stable/j.ctt2jc6s1

Broadhurst, R., Grabosky, P., Alazab, M., Bouhours, B., Chon, S., 2013. Organizations and cyber crime: an analysis of the nature of groups engaged in cyber crime. Int J Cyber Criminol 8, 1–20.

Bruschi, D., Martignoni, L., Monga, M., 2006. Detecting self-mutating malware using control-flow graph matching. In: Büschkes, R., Laskov, P. (Eds.), Detection of Intrusions and Malware & Vulnerability Assessment, vol. 4064. Springer, Berlin/Heidelberg, pp. 129–143.

Bureau, P., 2011. Same botnet, same guys, new code. In: Virus Bulletin International Conference, Barcelona, Spain, pp. 10–13.

Cao, X., Lu, Y., 2011. Social network analysis of a criminal hacker community. In: Nemati, H. (Ed.), Security and Privacy Assurance in Advancing Technologies: New Developments. IGI Global, pp. 160–173, http://www.unm.edu/~xinluo/papers/JCIS2011.pdf

Cesare, S., Xiang, Y., 2010. Classification of malware using structured control flow. In: 8th Australasian Symposium on Parallel and Distributed Computing, Brisbane, Australia, pp. 61–70.

Cesare, S., Xiang, Y., Zhou, W., 2014. Control flow-based malware variant detection. IEEE Trans. Dependable Secur. Comput. 11 (4), 307–317.

Chang, H., Atallah, M., 2002. Protecting software code by guards. In: Sander, T. (Ed.), Security and Privacy in Digital Rights Management, vol. 2320. Springer, Berlin/Heidelberg, pp. 125–141.

ClamAV. *An open source (GPL) antivirus engine*. Available: www.clamav.net

Cowley, S., 2012. Grum Takedown: '50% of Worldwide Spam is Gone', Available: http://money.cnn.com/2012/07/19/technology/grum-spam-botnet/

DataRescue, 2005. Using the Universal PE Unpacker Plug-In, Available: https://www.hex-rays.com/products/ida/support/tutorials/unpack_pe/unpacking.pdf

Dell, 2014. Dell Network Security Threat Report 2013, Available: http://www.sonicwall.com/app/projects/file_downloader/document_lib.php?t=WP&id=129.

Dinaburg, A., Royal, P., Sharif, M., Lee, W., 2008. Ether: malware analysis via hardware virtualization extensions. In: Proceedings of the 15th ACM conference on Computer and communications security, Alexandria, Virginia, USA, pp. 51–62.

Douglas, J., Ressler, R., Burgess, A., Hartman, C., 1986. Criminal profiling from crime scene analysis. Behav. Sci. Law 4, 401–421.

Ferrie, P., Szor, P., 2003. Hunting For Metamorphic, white paper Symantec Security Response, http://www.symantec.com/avcenter/reference/hunting.for.metamorphic.pdf

Fox-IT InTELL, 2014. Tilon/SpyEye2 Intelligence Report, Available: http://foxitsecurity.files.wordpress.com/2014/02/spyeye2_tilon_20140225.pdf

Frawley, W., Piatetsky-shapiro, G., Matheus, C., 1992. Knowledge discovery in databases: an overview. AI Mag. 13, 213–228.

Guo, F., Ferrie, P., Chiueh, T.C., 2008. A study of the packer problem and its solutions. In: Lippmann, R., Kirda, E., Trachtenberg, A. (Eds.), Recent Advances in Intrusion Detection, vol. 5230. Springer, Berlin/Heidelberg, pp. 98–115.

Hand, D.J., Mannila, H., Smyth, P., 2001. Principles of Data Mining. A Bradford Book.

Heavens, V.X., 2011. VX Heavens Site, 2/3, Available: http://vx.netlux.org/

IDA, 2011. IDA Pro, 5.5 ed.

Jacob, G., Debar, H., Filiol, E., 2008. Behavioral detection of malware: from a survey towards an established taxonomy. J. Comput. Virol. 4, 251–266.

Kang, M.G., Poosankam, P., Yin, H., 2007. Renovo: a hidden code extractor for packed executables. In: Proceedings of the 2007 ACM workshop on Recurring malcode, Alexandria, VA, USA, pp. 46–53.

Kolter, J., Maloof, M., 2004. Learning to detect malicious executables in the wild. In: The Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, NY, USA, pp. 470–478.

Kolter, J.Z., Maloof, M.A., 2006. Learning to detect and classify malicious executables in the wild. J. Mach. Learn. Res. 7, 2721–2744.

Lawton, G., 2002. Virus wars: fewer attacks, new threats. IEEE Comput. Soc. 35, 22–24.

Linn, C., Debray, S., 2003. Obfuscation of executable code to improve resistance to static disassembly. In: 10th ACM Conference on Computer and Communications Security, Washington, DC, USA, pp. 290–299.

Malan, D., Smith, M., 2005. Host-based detection of worms through peer-to-peer cooperation. In: The ACM Workshop on Rapid Malcode, Fairfax, VA, USA, pp. 72–80.

Martignoni, L., Christodorescu, M., Jha, S., 2007. OmniUnpack: fast, generic, and safe unpacking of malware. In: The 23rd Annual Computer Security Applications Conference, pp. 431–441.

MetaPHOR, 2010. W32.Simile, Available: http://www.symantec.com/security_response/writeup.jsp?docid=2002-030617-5423-99

Microsoft Developer Network, 2011. Windows API Functions, 12/10, Available: http://msdn.microsoft.com/en-us/

Moskovitch, R., Feher, C., Tzachar, N., Berger, E., Gitelman, M., Dolev, S., Elovici, Y., 2008. Unknown malcode detection using OPCODE representation. In: Esbjerg (Ed.), Proceedings of the 1st European Conference on Intelligence and Security Informatics. Springer-Verlag, Denmark, pp. 204–215.

O'Kane, P., Sezer, S., McLaughlin, K., 2011. Obfuscation: the hidden malware. Secur. Priv. IEEE 9, 41–47.

Oltsik, J., 2013. Malware and the State of Enterprise Security, July.

Orr, 2006. The Viral Darwinism of W32.Evol an In-Depth Analysis of a Metamorphic Engine.

Perriot, F., Ferrie, P., 2004. Principles and practise of x-raying. In: Virus Bulletin Conference, pp. 1–17.

Qinghua, Z., Reeves, D.S., 2007. MetaAware: identifying metamorphic malware. In: The Twenty-Third Annual Computer Security Applications Conference, Miami Beach, FL, pp. 411–420.

Rabek, J.C., Khazan, R.I., Lewandowski, S.M., Cunningham, R.K., 2003. Detection of injected, dynamically generated, and obfuscated malicious code. In: Presented at the Proceedings of the 2003 ACM workshop on Rapid Malcode, Washington, DC, USA.

Rad, B., Masrom, M., 2010. Metamorphic virus variants classification using opcode frequency histogram. In: Mastorakis, N., Mladenov, V., Bojkovic, Z. (Eds.), Latest Trends on Computers, vol. 1. WSEAS Press, pp. 147–155.

Rad, B., Masrom, M.,2011. Metamorphic virus detection in portable executables using opcodes statistical feature. In: International Conference on Advanced Science, Engineering and Information Technology. Kuala Lumpur, Malaysia, pp. 403–408.

Rogers, T., 2011. Tuts 4 You – Collection 2011, Available: https://tuts4you.com/download.php?list.52

Roundy, K.A., Miller, B.P., 2013. Binary-code obfuscations in prevalent packer tools. ACM Comput. Surv. 46, 1–32.

Royal, P., Halpin, M., Dagon, D., Edmonds, R., Lee, W., 2006. PolyUnpack: automating the hidden-code extraction of unpack-executing malware. In: The 22nd Annual Computer Security Applications Conference, Miami Beach, FL, USA, pp. 289–300.

RSA, 2011. The Current State of Cybercrime and What to Expect in 2011, RSA 2011 Cybercrime Trends Report.

Santos, I., Brezo, F., Nieves, J., Penya, Y., Sanz, B., Laorden, C., Bringas, P., 2010. Idea: opcode-sequence-based malware detection. In: Massacci, F., Wallach, D., Zannone, N. (Eds.), Engineering Secure Software and Systems, vol. 5965. Springer, Berlin Heidelberg, pp. 35–43.

Santos, I., Brezo, F., Ugarte-Pedrero, X., Bringas, P.G., 2013a. Opcode sequences as representation of executables for data-mining-based unknown malware detection. Inf. Sci. 231, 64–82.

Santos, I., Devesa, J., Brezo, F., Nieves, J., Bringas, P.G., 2013b. OPEM: A Static-Dynamic Approach for Machine-Learning-Based Malware Detection, Vol. 189, pp. 271–280, http://link.springer.com/chapter/10.1007/978-3-642-33018-6_28

Schultz, M.G., Eskin, E., Zadok, E., Stolfo, S.J., 2001. Data mining methods for detection of new malicious executables. In: IEEE Symposium on Security and Privacy, Oakland, CA, pp. 38–49.

Shankarapani, M., Kancherla, K., Ramammoorthy, S., Movva, R., Mukkamala, S., 2010. Kernel machines for malware classification and similarity analysis. In: The 2010 International Joint Conference on Neural Networks, Barcelona, pp. 1–6.

Sharif, M., Yegneswaran, V., Saidi, H., Porras, P., Lee, W., 2008. Eureka: a framework for enabling static malware analysis. In: Jajodia, S., Lopez, J. (Eds.), Computer Security – ESORICS 2008, vol. 5283. Springer, Berlin/Heidelberg, pp. 481–500.

SophosLabs, 2014. Security Threat Report 2013, Available. http://www.sophos.com/en-us/medialibrary/PDFs/other/sophossecuritythreatreport2013.pdf

SQLite, 2011. SQLite – Software Library That Implements SQL Database Engine, 3.7.7.1 ed. sqlite.org, Charlotte, NC.

Stepan, A., 2005. Defeating polymorphism: beyond emulation. In: The Virus Bulletin International Conference, pp. 40–48.

Stolfo, S., Wang, K., Li, W.-J., 2005. Fileprints: identifying file types by n-gram analysis. In: IEEE Workshop on Information Assurance United States Military Academy, West Point, NY.

Sung, A.H., Xu, J., Chavez, P., Mukkamala, S., 2004. Static analyzer of vicious executables (SAVE). In: 20th Annual Computer Security Applications Conference, Tucson, AZ, USA, pp. 326–334.

Symantec, 2011. Symantec Internet Security Threat Report: Trends for 2010, vol. 16, Available: https://www4.symantec.com/mktginfo/downloads/21182883_GA_REPORT_ISTR_Main-Report_04-11_HI-RES.pdf

Symantec, 2012. Symantec Intelligence Report, February. Available: http://www.symantec.com/content/en/us/enterprise/other_resources/b-intelligence_report_02_2012.en-us.pdf

Symantec Corporation, 2014. Internet Security Threat Report, vol. 19, April.

Symantec Enterprise Security, 1997. Understanding heuristics: symantec's bloodhound technology. Symantec White Paper Series, Vol. XXXIV., pp. 1–17, http://www.symantec.com/avcenter/reference/heuristc.pdf

Symantec Enterprise Security, 2009. Symantec Global Internet Security Threat Report Trends for 2008. Symantec Enterprise Security.

Symantec Enterprise Security, 2010. Symantec Internet Security Threat Report: Trends for 2009. Symantec Enterprise Security.

Tamada, H., Okamoto, K., Nakamura, M., Monden, A., Matsumoto, K.I., 2004. Dynamic software birthmarks to detect the theft of windows applications. International Symposium on Future Software Technology (Vol. 20, No. 22), http://www.sea.jp/Events/isfst/ISFST2004/CDROM04/Presented04/2P1-T1/isfst2004_J355.pdf.

Tang, K., Zhou, M.-T., Zuo, Z.-H., 2010. An enhanced automated signature generation algorithm for polymorphic malware detection. J. Electron. Sci. Technol. 8, 114–121.

Thrilling, S., 2008. Project Green Bay – Calling a Blitz on Packers, Available: http://eval.symantec.com/mktginfo/enterprise/articles/b-ciodigest_october08_upload.pdf

UNODC, 2013. Comprehensive Study on Cybercrime, Available: http://www.unodc.org/documents/organized-crime/UNODC_CCPCJ_EG.4_2013/CYBERCRIME_STUDY_210213.pdf

Venkatraman, S., 2009. Autonomic context-dependent architecture for malware detection. In: Presented at the e-Tech 2009 International Conference on e-Technology, Singapore, pp. 2927–2947.

Wang, C., Pang, J., Zhao, R., Fu, W., Liu, X., 2009. Malware detection based on suspicious behavior identification. In: First International Workshop on Education Technology and Computer Science, Wuhan, Hubei, China, pp. 198–202.

Xu, J.-Y., Sung, A.H., Chavez, P., Mukkamala, S., 2004. Polymorphic malicious executable scanner by API sequence analysis. In: Presented at the Proceedings of the Fourth International Conference on Hybrid Intelligent Systems.

Xu, J., Sung, A., Mukkamala, S., Liu, Q., 2007. Obfuscated malicious executable scanner. J. Res. Pract. Inf. Technol. 39, 181–197, https://www.acs.org.au/_data/assets/pdf_file/0003/15375/JRPIT39.3.181.pdf

Yanfang, Y., Tao, L., Qingshan, J., Youyu, W., 2010. CIMDS: adapting postprocessing techniques of associative classification for malware detection. IEEE Trans. Syst. Man Cybern. C: Appl. Rev. 40, 298–307.

Ye, Y., Wang, D., Li, T., Ye, D., 2007. IMDS: intelligent malware detection system. In: Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining, San Jose, California, USA, pp. 1043–1047.

Dr Mamoun Alazab is a Research Fellow at the Australian National University (ANU) and the Co-founder of the ANU Cybercrime Observatory. He is a computer security researcher & practitioner with industry, academic, teaching and research experience. He completed his PhD degree in IT Security from the Federation University of Australia. He is an active contributor to the academic as well as broader international community working on Information Security issues and for the last decade has contributed to research and teaching especially in the area of cyber security and cybercrime. He has worked closely with government and industry on many projects, including IBM, the Australian Federal Police (AFP), Australian Communications and Media Authority, Westpac, and Attorney General's Department. He has also consulted for the United Nations (UN) Office on Drugs and Crime (UNODC), and other industry groups.